

练习2：创建多个分支，进行分支合并操作

2025年9月4日 11:20

练习 2：创建多个分支并进行分支合并

前提

你已经完成了 **练习 1**（有一个本地 Git 仓库，并且里面至少有一个文件和几次提交）。

步骤 1：确认当前分支

默认情况下，Git 仓库会有一个主分支（main 或 master）。

```
git branch
```

输出示例：

```
* main
```

(* 表示你当前所在的分支)

步骤 2：创建一个新分支

假设我们要开发一个新功能，可以创建一个分支 feature-login：

```
git branch feature-login
```

查看分支列表：

```
git branch
```

输出：

```
* main
```

```
feature-login
```

步骤 3：切换到新分支

```
git checkout feature-login
```

输出：

```
Switched to branch 'feature-login'
```

确认：

```
git branch
```

输出：

```
main
```

```
* feature-login
```

步骤 4：在新分支上修改代码

编辑你的文件（比如 README.md），加一行内容：

这是在 feature-login 分支上添加的内容。

保存后，提交修改：

```
git add .
git commit -m "在 feature-login 分支上添加登录功能说明"
```

步骤 5：切换回 main 分支

```
git checkout main
```

查看文件内容，你会发现 main 分支的文件没有新加那一行。这说明 feature-login 分支的修改是独立的。

步骤 6：合并分支

现在把 feature-login 分支的修改合并到 main：

```
git merge feature-login
```

如果没有冲突，输出会像这样：

```
Updating alb2c3d..e4f5g6h
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

此时 main 分支已经包含 feature-login 的修改。

步骤 7：删除分支（可选）

合并完成后，可以删除已经用过的分支：

```
git branch -d feature-login
```

步骤 8：创建另一个分支并模拟冲突（进阶）

再创建一个分支：

```
git checkout -b feature-register
```

修改 README.md，比如写：

这是在 feature-register 分支上添加的内容。

提交：

```
git add .
git commit -m "在 feature-register 分支上添加注册功能说明"
```

切回 main，在同一个文件的同一行写别的内容，然后提交。

再执行：

```
git merge feature-register
```

你可能会遇到 **冲突**，Git 会提示：

```
CONFLICT (content): Merge conflict in README.md
```

打开 README.md 会看到类似：

```
<<<<<< HEAD
```

这是 main 分支上的修改

```
=====
```

这是在 feature-register 分支上添加的内容。

```
>>>>>> feature-register
```

你需要手动修改，保留需要的内容，然后执行：

```
git add README.md
```

```
git commit
```

这样冲突就解决了。

☑ 总结

- **创建分支**：git branch 分支名
- **切换分支**：git checkout 分支名 或 git switch 分支名
- **合并分支**：git merge 分支名
- **删除分支**：git branch -d 分支名
- **冲突处理**：手动编辑文件 → git add → git commit

好的，以下是为 **练习 2** 创建的 **分支操作清单 (Checklist)**，你可以用来引导自己和队友完成分支创建、切换、合并等操作。

☑ Git 分支操作清单

1. 初始化 Git 仓库

- 如果你还没有仓库，先创建一个本地仓库：

```
git init
```

2. 确认当前分支

- 确认你当前所在的分支：

```
git branch
```

3. 创建并切换到新分支

- 创建一个新分支 feature-login：

```
git branch feature-login
```

- 切换到 feature-login 分支：

```
git checkout feature-login
```

4. 在新分支上进行修改

- 修改文件，比如 README.md，并添加一行内容：

这是在 feature-login 分支上添加的内容。

- 保存文件，添加并提交修改：

```
git add .  
git commit -m "在 feature-login 分支上添加登录功能说明"
```

5. 切换回主分支

- 切换回 main 分支：

```
git checkout main
```

6. 查看当前文件 (main 分支)

- 确认 main 分支上的文件没有 feature-login 分支的修改。

7. 合并 feature-login 分支到 main

- 执行合并操作：

```
git merge feature-login
```

- 确认合并是否成功，并查看修改。

8. 删除已合并的分支

- 删除已经合并的 feature-login 分支：

```
git branch -d feature-login
```

9. 创建另一个分支并进行冲突模拟

- 创建并切换到 feature-register 分支：

```
git checkout -b feature-register
```

- 修改 README.md 文件并提交：

```
echo "这是在 feature-register 分支上添加的内容。" >> README.md  
git add .  
git commit -m "在 feature-register 分支上添加注册功能说明"
```

10. 回到 main 分支并修改文件

- 切换回 main 分支：

```
git checkout main
```

- 修改 README.md 文件，增加不同内容：

```
echo "这是在 main 分支上的修改。" >> README.md
git add .
git commit -m "在 main 分支上修改 README"
```

11. 尝试合并并解决冲突

- 合并 feature-register 分支到 main:

```
git merge feature-register
```

- 如果有冲突, Git 会提示你进行冲突解决:
 - 打开冲突文件 (如 README.md), 手动解决冲突 (保留需要的内容)。
 - 标记解决后的文件:

```
git add README.md
git commit
```

12. 查看提交历史

- 查看分支的提交历史, 确认所有操作:

```
git log --oneline --graph --all
```

13. 练习总结

- 你已成功:
 - 创建了多个分支。
 - 在分支上进行修改并提交。
 - 合并分支并解决冲突。
 - 删除了不再需要的分支。

练习小提示

- 每次创建分支时, 先确保你已经切换到目标分支。
- 在进行合并操作前, 建议先执行 `git status`, 确认是否有未提交的更改。
- 遇到冲突时, 仔细检查每个冲突的部分, 确保最终的修改符合预期。